

Intermediate Code Generation

a.y. 2025-2026

1 / 85

Compiler front-end

Phases of the front-end:

- Lexical analysis
- Parsing
- Semantic analysis (static checking: operators are applied to compatible operands; return-statement or break-statement enclosed within a while-statement; ecc.)
- Intermediate code generation

Semantic analysis and intermediate code generation can be specified by syntax-directed translations

Many translation schemes can be implemented during parsing, all of them can be implemented by creating an abstract syntax tree and then walking it

2 / 85

Intermediate representation

Many possibilities:

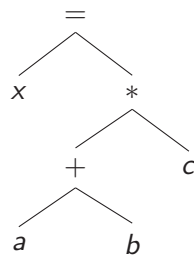
- Graph-like structures: abstract syntax trees or directed acyclic graphs
- Three-address code: $x = y \text{ op } z$
- Another language (C is favoured by its flexibility)

3 / 85

Three-address code

Linearised representation of syntax trees

Example:



Provide temporary names for interior nodes

$$\begin{aligned}
 t_1 &= a + b \\
 t_2 &= t_1 * c \\
 x &= t_2
 \end{aligned}$$

Desirable representation for target-code generation and optimization

4 / 85

Three-address code

A variety of instructions allowed:

- $a1 = a2 \text{ op } a3$
- $a1 = \text{op } a2$
- $a1 = a2$
- $a1 = a2[a3]$
- $a1[a2] = a3$
- ...
- *goto* L
- *if a goto* L
- ...

5 / 85

Intermediate code

For instance

```
if ( x < 100 || x > 200 && x != y ) x=0;
```

Can be translated to

```
IF x < 100 GOTO L2
GOTO L3
L3: IF x > 200 GOTO L4
GOTO L1
L4: IF x != y GOTO L2
GOTO L1
L2: x = 0
L1:
```

6 / 85

Intermediate code

For instance

```
if ( x < 100 || x > 200 && x != y ) x=0;
```

Can be translated to

```
      IF x < 100 GOTO L2
      GOTO L3
L3:   IF x > 200 GOTO L4
      GOTO L1
L4:   IF x != y GOTO L2
      GOTO L1
L2:   x = 0
L1:
```

Or to

```
      IF x < 100 GOTO L2
      IFFALSE x > 200 GOTO L1
      IFFALSE x != y GOTO L1
L2:   x = 0
L1:
```

Syntax-directed translation of expressions

$$S \rightarrow id = E$$

$$E \rightarrow E_1 + E_2$$

$$E \rightarrow -E_1$$

$$E \rightarrow (E_1)$$

$$E \rightarrow id$$

Syntax-directed translation of expressions

Goal: emit three-address code

Attributes and auxiliary functions:

- $E.addr$ denotes the temporary holding the value of E
- $S.code$, $E.code$ denote the code emitted for S and E
- $gen(str)$ emits the string str
- $newtemp()$ generates a new temporary
- Symbol ' $\triangleright$ ' stands for the concatenation of intermediate-code fragments (overloaded to deal with labels)

8 / 85

Syntax-directed translation of expressions

$$\begin{array}{ll}
 S \rightarrow id = E & S.code = E.code \triangleright \\
 & \quad gen(table.get(id) '=' E.addr) \\
 \\
 E \rightarrow E_1 + E_2 & E.addr = newtemp() \\
 & E.code = E_1.code \triangleright E_2.code \triangleright \\
 & \quad gen(E.addr '=' E_1.addr '+' E_2.addr)
 \end{array}$$

9 / 85

Syntax-directed translation of expressions

$E \rightarrow -E_1$	$E.addr = newtemp()$ $E.code = E_1.code \triangleright$ $gen(E.addr '=' '-' E_1.addr)$
$E \rightarrow (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$E \rightarrow id$	$E.addr = table.get(id)$ $E.code = ' '$

10 / 85

Syntax-directed translation of expressions

TRAINING: $a = b + -c$

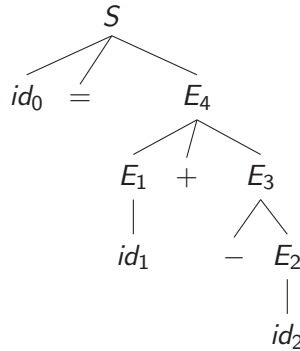
- The grammar is ambiguous, to keep it succinct and focus on code generation
- Guess the 'right' tree for $a = b + -c$
- What's the code then?

11 / 85

Syntax-directed translation of expressions

TRAINING: $a = b + -c$

Indexing terminals and non-terminals for convenience, the derivation tree is



Attributes can be computed during bottom-up parsing

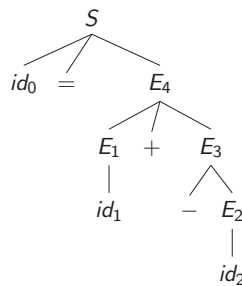
12 / 85

Syntax-directed translation of expressions

TRAINING: $a = b + -c$

First reduction: $E_1 \rightarrow id_1$

Relevant rules: $E_1.addr = table.get(id_1)$; $E_1.code = ''$



Evaluation: $E_1.addr = b$; $E_1.code = ''$

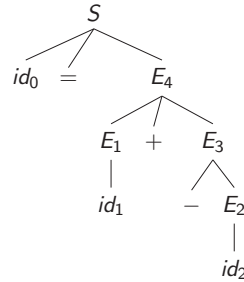
13 / 85

Syntax-directed translation of expressions

TRAINING: $a = b + -c$

Second reduction: $E_2 \rightarrow id_2$

Relevant rules: $E_2.addr = table.get(id_2)$; $E_2.code = ' '$



Evaluation: $E_2.addr = c$; $E_2.code = ' '$

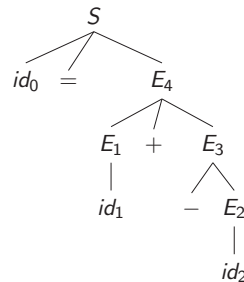
14 / 85

Syntax-directed translation of expressions

TRAINING: $a = b + -c$

Third reduction: $E_3 \rightarrow -E_2$

Relevant rules: $E_3.addr = newtemp()$;
 $E_3.code = E_2.code \triangleright gen(E_3.addr \text{ '=' } ' - ' E_2.addr)$



Evaluation: $E_3.addr = t_1$; $E_3.code = [t_1 = -c]$

15 / 85

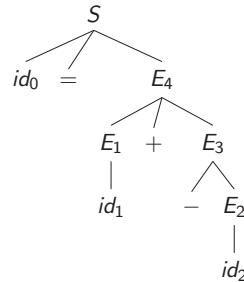
Syntax-directed translation of expressions

TRAINING: $a = b + -c$

Fourth reduction: $E_4 \rightarrow E_1 + E_3$

Relevant rules: $E_4.addr = newtemp();$

$E_4.code = E_1.code \triangleright E_3.code \triangleright gen(E_4.addr '=' E_1.addr '+' E_3.addr)$



Evaluation: $E_4.addr = t_2$; $E_4.code = [t_1 = -c] \triangleright [t_2 = b + t_1]$

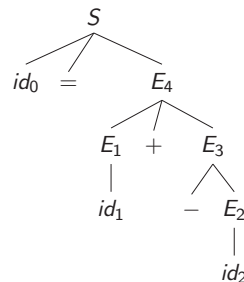
16 / 85

Syntax-directed translation of expressions

TRAINING: $a = b + -c$

Last reduction: $S \rightarrow id_0 = E_4$

Relevant rule: $S.code = E_4.code \triangleright gen(table.get(id_0) '=' E_4.addr)$



Evaluation: $S.code = [t_1 = -c] \triangleright [t_2 = b + t_1] \triangleright [a = t_2]$

17 / 85

Control flow statements

$$P \rightarrow S$$

$$S \rightarrow S_1 S_2$$

$$S \rightarrow \text{if } (B) S_1$$

$$S \rightarrow \text{while } (B) S_1$$

$$\dots$$

$$B \rightarrow \text{true}$$

$$B \rightarrow \text{false}$$

$$B \rightarrow \text{not } B$$

$$B \rightarrow E_1 \text{ rel } E_2$$

$$B \rightarrow B_1 \parallel B_2$$

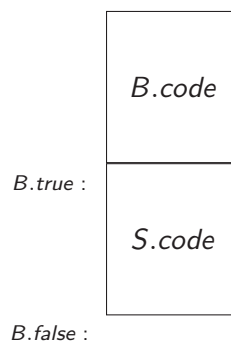
$$B \rightarrow B_1 \ \&\& \ B_2$$

$$\dots$$

18 / 85

Control flow statements

EXAMPLE: TRANSLATION OF $\text{if}(B)S$



19 / 85

Control flow statements

Attributes for statements and boolean conditions:

- *S.next*, inherited, labels the beginning of the code that must be executed after *S* is finished
- *B.true*, inherited, labels the beginning of the code that must be executed if *B* is true
- *B.false*, inherited, labels the beginning of the code that must be executed if *B* is false
- *S.code*, synthesised, is the sequence of intermediate-code steps that implements the statement *S* and ends with a jump to *S.next*
- *B.code* is the sequence of intermediate-code steps that implements the boolean condition *B* and jumps to *B.true* if *B* is true, and to *B.false* otherwise. *B.code* inherits *B.true* and *B.false* and synthesises all the rest

20 / 85

Control flow statements

Auxiliary functions:

- *newlabel()* generates a new label at each invocation
- *label(L)* attaches label *L* to the next three-address instruction to be generated

21 / 85

Control flow statements

BOOLEAN CONDITIONS

$$B \rightarrow \text{true} \quad B.\text{code} = \text{gen}(\text{GOTO } B.\text{true})$$

$$B \rightarrow \text{false} \quad B.\text{code} = \text{gen}(\text{GOTO } B.\text{false})$$

The actual position of labels $B.\text{true}$ and $B.\text{false}$ is established at the translation of statements

For instance, at the translation of ' $\text{if}(B)S$ ', we generate the label $B.\text{true}$ and emit the code

$$B.\text{code} \triangleright \text{label}(B.\text{true}) \triangleright S.\text{code}$$

22 / 85

Control flow statements

Notice:

Booleans have two distinct roles:

- Alter the flow of control (boolean conditions, like in $\text{if}(a \ \&\& \ b) \ x = 0$)
- Compute logical values (boolean expressions, as right-hand sides of assignments, like in $x = a \ \&\& \ b$)

What we consider here is their translation in the context of control flow statements

Seen as expressions, they are translated analogously to what is done for arithmetic expressions, with logical operators rather than arithmetic operators

The grammar can have distinct nonterminals to distinguish these two roles

23 / 85

Control flow statements

BOOLEAN CONDITIONS

$$B \rightarrow \text{not } B_1 \quad \begin{array}{l} B_1.\text{true} = B.\text{false} \\ B_1.\text{false} = B.\text{true} \\ B.\text{code} = B_1.\text{code} \end{array}$$

$$B \rightarrow E_1 \text{ rel } E_2 \quad \begin{array}{l} B.\text{code} = E_1.\text{code} \triangleright E_2.\text{code} \triangleright \\ \text{gen}(\text{IF } E_1.\text{addr rel op } E_2.\text{addr GOTO } B.\text{true}) \triangleright \\ \text{gen}(\text{GOTO } B.\text{false}) \end{array}$$

Control flow statements

BOOLEAN CONDITIONS

$$B \rightarrow B_1 \parallel B_2 \quad \begin{array}{l} B_1.\text{true} = B.\text{true} \\ B_1.\text{false} = \text{newlabel}() \\ B_2.\text{true} = B.\text{true} \\ B_2.\text{false} = B.\text{false} \\ B.\text{code} = B_1.\text{code} \triangleright \\ \text{label}(B_1.\text{false}) \triangleright B_2.\text{code} \end{array}$$

Control flow statements

BOOLEAN CONDITIONS

$$\begin{aligned}
 B \rightarrow B_1 \ \&\& \ B_2 \quad & B_1.true = newlabel() \\
 & B_1.false = B.false \\
 & B_2.true = B.true \\
 & B_2.false = B.false \\
 & B.code = B_1.code \triangleright \\
 & \quad label(B_1.true) \triangleright B_2.code
 \end{aligned}$$

26 / 85

Control flow statements

$$\begin{aligned}
 P \rightarrow S \quad & S.next = newlabel(); \\
 & P.code = S.code \triangleright label(S.next)
 \end{aligned}$$

$S.next$ is the label of the instruction executed when S is over

If after performing some statements in S we need to jump at another location then we must set up an explicit GOTO instruction

27 / 85

Control flow statements

$$\begin{aligned} S \rightarrow S_1 S_2 \quad & S_1.next = newlabel() \\ & S_2.next = S.next \\ & S.code = S_1.code \triangleright label(S_1.next) \triangleright S_2.code \end{aligned}$$

28 / 85

Control flow statements

$$\begin{aligned} S \rightarrow \text{if } (B) S_1 \quad & B.true = newlabel() \\ & B.false = S.next \\ & S_1.next = S.next \\ & S.code = B.code \triangleright label(B.true) \triangleright S_1.code \end{aligned}$$

29 / 85

Control flow statements

$$\begin{aligned}
 S \rightarrow \text{while } (B) S_1 \quad & B.true = \text{newlabel}() \\
 & B.false = S.next \\
 & S_1.next = \text{newlabel}() \\
 & S.code = \text{label}(S_1.next) \triangleright B.code \triangleright \\
 & \quad \text{label}(B.true) \triangleright S_1.code \triangleright \\
 & \quad \text{gen}(\text{GOTO } S_1.next)
 \end{aligned}$$

30 / 85

Control flow statements

TRAINING: $\text{if } ((a = b) \parallel (c = d)) \text{ if } ((e = f) \&\& (g = h)) \text{ skip}$

Code generation for

$$\text{if } ((a = b) \parallel (c = d)) \text{ if } ((e = f) \&\& (g = h)) \text{ skip}$$

Where the statement *skip* (meaning “do nothing”) has been added to the grammar to simplify generation

The grammar is then extended with

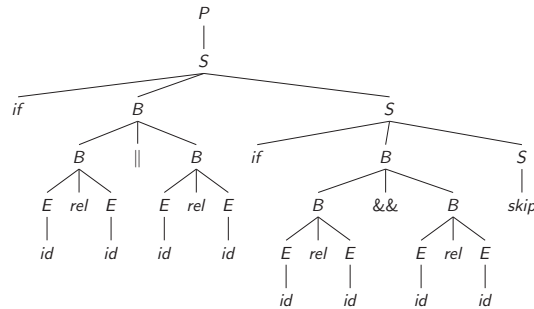
$$S \rightarrow \text{skip} \quad S.code = ' '$$

31 / 85

Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

Omitting leaves for parentheses, the derivation tree is



$S.next$, $B.true$, $B.false$ are inherited attributes

We run a first visit of the tree to compute them

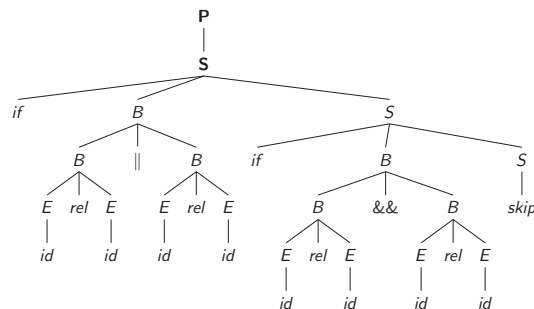
32 / 85

Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

Relevant production: $P \rightarrow S$

Relevant rule: $S.next = newlabel()$



Evaluation:

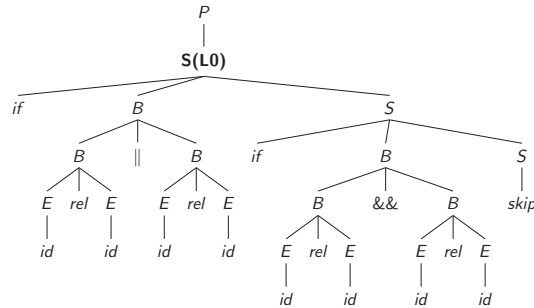
$S.next = newlabel() = L0$

33 / 85

Control flow statements

TRAINING: $\text{if } ((a = b) \parallel (c = d)) \text{ if } ((e = f) \&\& (g = h)) \text{ skip}$

We keep information at nodes and continue the evaluation



34 / 85

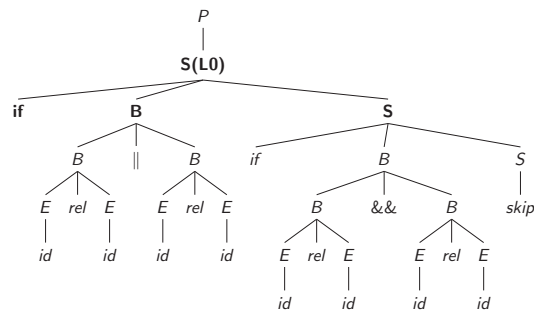
Control flow statements

TRAINING: $\text{if } ((a = b) \parallel (c = d)) \text{ if } ((e = f) \&\& (g = h)) \text{ skip}$

Next production: $S \rightarrow \text{if } (B) S_1$

Relevant rules:

$B.\text{true} = \text{newlabel}(); B.\text{false} = S.\text{next}; S_1.\text{next} = S.\text{next}$



Evaluation:

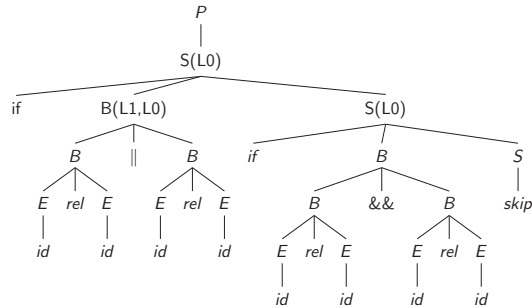
$B.\text{true} = L1; B.\text{false} = L0; S_1.\text{next} = L0$

35 / 85

Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

Up to this point we have



36 / 85

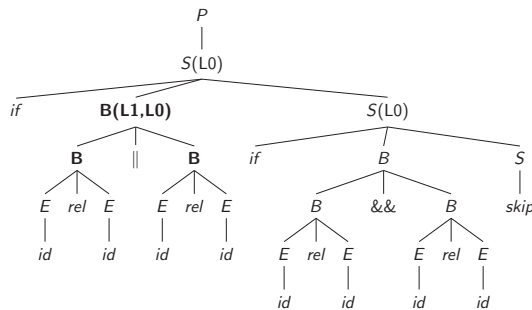
Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

Next production: $B \rightarrow B_1 \parallel B_2$

Relevant rules: $B_1.true = B.true$; $B_1.false = newlabel()$;

$B_2.true = B.true$; $B_2.false = B.false$;



Evaluation:

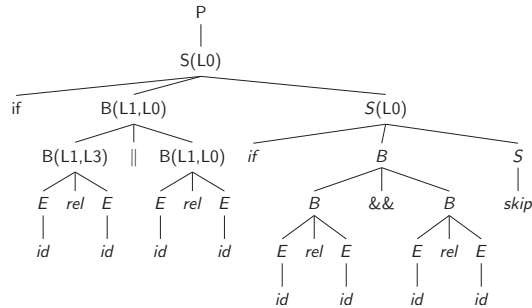
$B_1.true = L1$; $B_1.false = L3$; $B_2.true = L1$; $B_2.false = L0$

37 / 85

Control flow statements

TRAINING: $if ((a = b) \parallel (c = d)) \text{ if } ((e = f) \&\& (g = h)) \text{ skip}$

Up to now



38 / 85

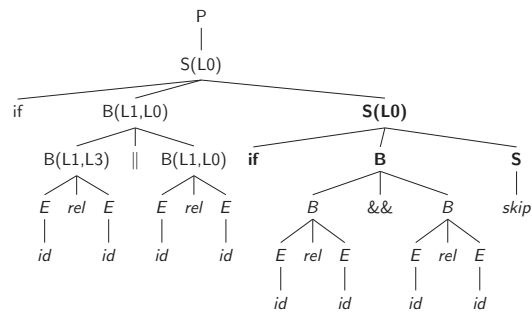
Control flow statements

TRAINING: $if ((a = b) \parallel (c = d)) \text{ if } ((e = f) \&\& (g = h)) \text{ skip}$

Next production: $S \rightarrow if (B) S_1$

Relevant rules:

$B.true = newlabel(); B.false = S.next; S_1.next = S.next$



Evaluation:

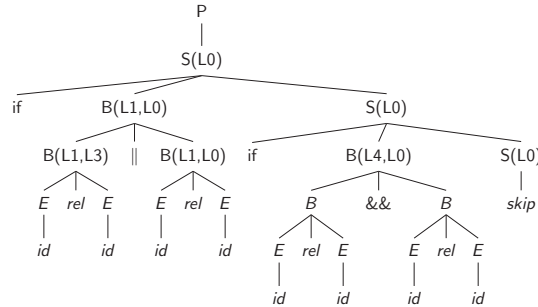
$B.true = L4; B.false = L0; S_1.next = L0$

39 / 85

Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

Up to now



40 / 85

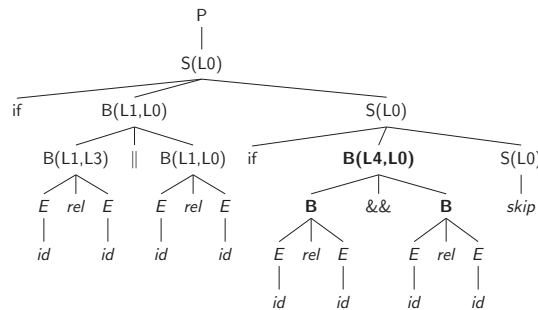
Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

Next production: $B \rightarrow B_1 \&\& B_2$

Relevant rules: $B_1.true = newlabel(); B_1.false = B.false;$

$B_2.true = B.true; B_2.false = B.false;$



Evaluation:

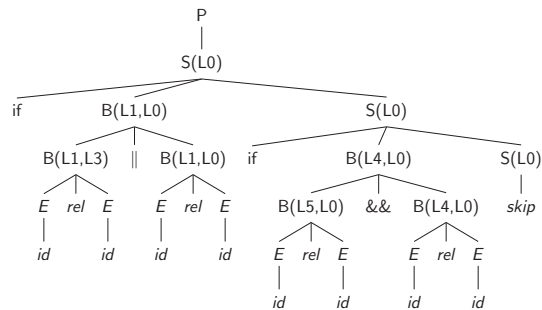
$B_1.true = L5; B_1.false = L0; B_2.true = L4; B_2.false = L0;$

41 / 85

Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

That is all for inherited attributes



42 / 85

Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

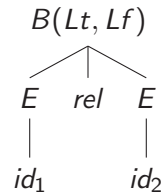
Code attributes now

43 / 85

Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

For each tree of the shape



By the rule associated with $B \rightarrow E \text{ rel } E$, we get

IF $id_1 \text{ relop } id_2$ *GOTO* Lt
GOTO Lf

44 / 85

Control flow statements

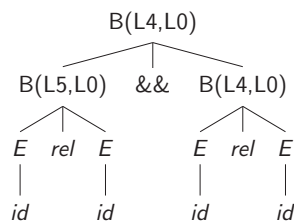
TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

For

$(e = f) \&\& (g = h)$

By the conjunction-rule, we generate

$B_1.\text{code} \triangleright \text{label}(B_1.\text{true}) \triangleright B_2.\text{code}$



IF $e = f$ *GOTO* $L5$
GOTO $L0$
 $L5 :$ *IF* $g = h$ *GOTO* $L4$
GOTO $L0$

45 / 85

Control flow statements

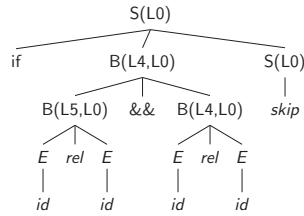
TRAINING: $\text{if } ((a = b) \parallel (c = d)) \text{ if } ((e = f) \&\& (g = h)) \text{ skip}$

For

$\text{if } ((e = f) \&\& (g = h)) \text{ skip}$

By the if-rule, we generate

$B.\text{code} \triangleright \text{label}(B.\text{true}) \triangleright S.\text{code}$



$\text{IF } e = f \text{ GOTO L5}$
 GOTO L0
 $\text{L5 : IF } g = h \text{ GOTO L4}$
 GOTO L0
 L4 :

Control flow statements

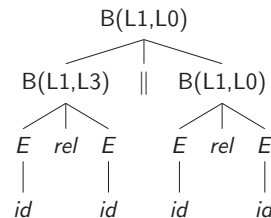
TRAINING: $\text{if } ((a = b) \parallel (c = d)) \text{ if } ((e = f) \&\& (g = h)) \text{ skip}$

For

$(a = b) \parallel (c = d)$

By the disjunction-rule, we generate

$B_1.\text{code} \triangleright \text{label}(B_1.\text{false}) \triangleright B_2.\text{code}$



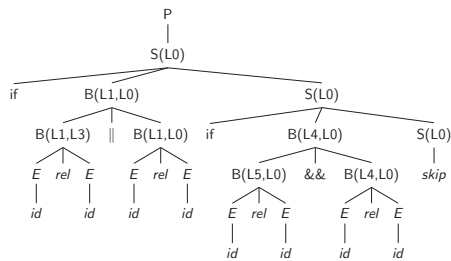
$\text{IF } a = b \text{ GOTO L1}$
 GOTO L3
 $\text{L3 : IF } c = d \text{ GOTO L1}$
 GOTO L0

Control flow statements

TRAINING: *if* $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*

For all of it

if $((a = b) \parallel (c = d))$ *if* $((e = f) \&\& (g = h))$ *skip*



IF $a = b$ *GOTO* L1

GOTO L3

L3 : *IF* $c = d$ *GOTO* L1

GOTO L0

L1 : *IF* $e = f$ *GOTO* L5

GOTO L0

L5 : *IF* $g = h$ *GOTO* L4

GOTO L0

L0, L4 :

48 / 85

Control flow statements

BOOLEAN CONDITIONS

The above translation generates codes like the one on the left

IF $x < 100$ *GOTO* L2

GOTO L3

L3: *IF* $x > 200$ *GOTO* L4

GOTO L1

L4: *IF* $x \neq y$ *GOTO* L2

GOTO L1

L2: $x = 0$

L1:

IF $x < 100$ *GOTO* L2

IFFALSE $x > 200$ *GOTO* L1

IFFALSE $x \neq y$ *GOTO* L1

L2: $x = 0$

L1:

We can organize the translation to get the code on the right instead,
and so avoid redundant gotos

49 / 85

Control flow statements

FALL TECHNIQUE: AVOIDING REDUNDANT GOTOS

Under the hypothesis that if the boolean condition is true then the next instruction to execute is always the next in sequence, we can reformulate the SDD for

$$P \rightarrow S$$

$$S \rightarrow \text{if } (B) S_1 \mid \text{while } (B) S_1$$

...

$$B \rightarrow \text{true} \mid \text{false} \mid \text{not } B \mid E_1 \text{ rel } E_2 \mid B_1 \parallel B_2 \mid B_1 \ \&\& \ B_2$$

...

Using a special label “fall” which does not generate gotos (just fall down to the next instruction in sequence)

Control flow statements

FALL TECHNIQUE: AVOIDING REDUNDANT GOTOS

$$P \rightarrow S$$

$$\begin{aligned} S.next &= \text{newlabel}(); \\ P.code &= S.code \triangleright \text{label}(S.next) \end{aligned}$$

$$S \rightarrow \text{if } (B) S_1$$

$$\begin{aligned} B.true &= \text{fall} \\ B.false &= S.next \\ S_1.next &= S.next \\ S.code &= B.code \triangleright S_1.code \end{aligned}$$

Control flow statements

FALL TECHNIQUE: AVOIDING REDUNDANT GOTOS

$S \rightarrow \text{while } (B) S_1$	$B.\text{true} = \text{fall}$ $B.\text{false} = S.\text{next}$ $S_1.\text{next} = \text{newlabel}()$ $S.\text{code} =$ $\quad \text{label}(S_1.\text{next}) \triangleright B.\text{code} \triangleright$ $\quad S_1.\text{code} \triangleright \text{gen}(\text{GOTO } S_1.\text{next})$
$B \rightarrow \text{true}$	$\text{if } B.\text{true} \neq \text{fall}$ $\text{then gen}(\text{GOTO } B.\text{true})$
$B \rightarrow \text{false}$	$\text{if } B.\text{false} \neq \text{fall}$ $\text{then gen}(\text{GOTO } B.\text{false})$

52 / 85

Control flow statements

FALL TECHNIQUE: AVOIDING REDUNDANT GOTOS

$B \rightarrow \text{not } B_1$	$B_1.\text{true} = B.\text{false}$ $B_1.\text{false} = B.\text{true}$ $B.\text{code} = B_1.\text{code}$
$B \rightarrow E_1 \text{ rel } E_2$	$B.\text{code} = E_1.\text{code} \triangleright E_2.\text{code} \triangleright st$

Where, for $\text{test} = E_1.\text{addr} \text{ relop } E_2.\text{addr}$

```

st = if B.true ≠ fall and B.false ≠ fall
    then gen(IF test GOTO B.true) ▷ gen(GOTO B.false)
    elseif B.true ≠ fall
        then gen(IF test GOTO B.true)
    elseif B.false ≠ fall
        then gen(IFFALSE test GOTO B.false)
    else gen(' ')

```

53 / 85

Control flow statements

FALL TECHNIQUE: AVOIDING REDUNDANT GOTOS

$$B \rightarrow B_1 \parallel B_2$$

$B_1.true = \text{if } B.true \neq fall$
 $\quad \text{then } B.true$
 $\quad \text{else } newlabel()$
 $B_1.false = fall$
 $B_2.true = B.true$
 $B_2.false = B.false$
 $B.code =$
 $\quad \text{if } B.true \neq fall$
 $\quad \text{then } B_1.code \triangleright B_2.code$
 $\quad \text{else } B_1.code \triangleright B_2.code \triangleright label(B_1.true)$

When $B.true = fall$, $B_1.true$ makes the control jump over $B_2.code$ to get to the next instruction after B

Control flow statements

FALL TECHNIQUE: AVOIDING REDUNDANT GOTOS

$$B \rightarrow B_1 \&\& B_2$$

$B_1.true = fall$
 $B_1.false = \text{if } B.false \neq fall$
 $\quad \text{then } B.false$
 $\quad \text{else } newlabel()$
 $B_2.true = B.true$
 $B_2.false = B.false$
 $B.code =$
 $\quad \text{if } B.false \neq fall$
 $\quad \text{then } B_1.code \triangleright B_2.code$
 $\quad \text{else } B_1.code \triangleright B_2.code \triangleright label(B_1.false)$

When $B.false = fall$, $B_1.false$ makes the control jump over $B_2.code$ to get to the next instruction after B

Typing Array Elements

Two main strategies to store arrays:

Row-major order. Change the rightmost index: for a 2×3 matrix A , store:

$A[0][0]$
 $A[0][1]$
 $A[0][2]$
 $A[1][0]$
 $A[1][1]$
 $A[1][2]$

This implies that for the code

```
for  $i = 0$  to 1  
  for  $j = 0$  to 3  
     $A[i][j] = 1$ 
```

The assignment steps through memory in sequential order

56 / 85

Typing Array Elements

Column-major order. Change the leftmost index: for the same 2×3 matrix A , store:

$A[0][0]$
 $A[1][0]$
 $A[0][1]$
 $A[1][1]$
 $A[0][2]$
 $A[1][2]$

57 / 85

Typing Array Elements

- A matrix 2×3 is seen as
An array of 2 arrays of 3 integers
- We assign to such a matrix the type "*array(2, array(3, integer))*"

58 / 85

Typing Array Elements

$$T \rightarrow BC$$

$$B \rightarrow \text{int} \mid \text{float}$$

$$C \rightarrow [\text{num}] C \mid \epsilon$$

Attributes:

- $T.t$, synthesized, to represent the computed type
- $B.t$, synthesized, to represent the basic type
- $C.b$, inherited, to pass the basic type down the tree
- $C.t$, synthesized, to accumulate the result after applying the operator *array*

59 / 85

Typing Array Elements

$$T \rightarrow BC$$

$$T.t = C.t$$

$$C.b = B.t$$

$$B \rightarrow \text{int}$$

$$B.t = \text{integer}$$

$$B \rightarrow \text{float}$$

$$B.t = \text{float}$$

$$C \rightarrow [\text{num}] C_1$$

$$C.t = \text{array}(\text{num.lexval}, C_1.t)$$

$$C_1.b = C.b$$

$$C \rightarrow \epsilon$$

$$C.t = C.b$$

60 / 85

Typing Array Elements

TRAINING: typing $\text{int}[2][3]$

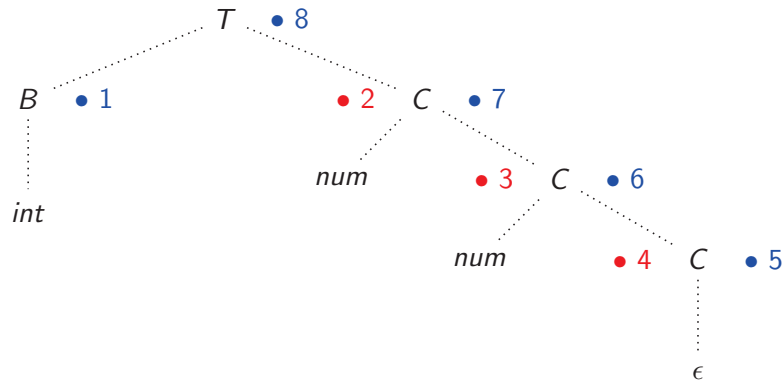
Derive the type of $\text{int}[2][3]$

61 / 85

Typing Array Elements

TRAINING: typing $int[2][3]$

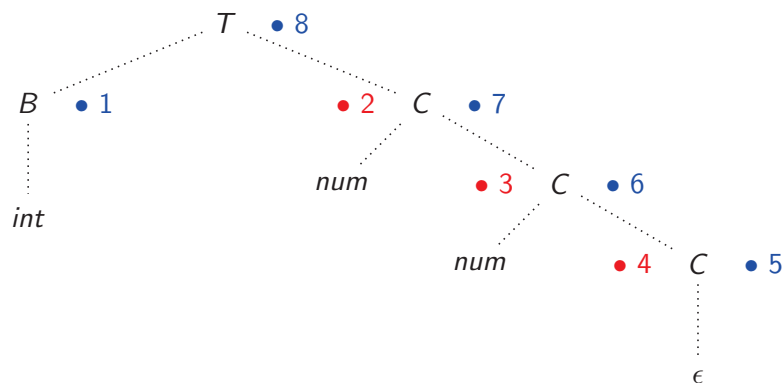
Annotated parse tree for $int[2][3]$ and evaluation order (synthesised attribute nodes in blue, inherited attribute nodes in red)



62 / 85

Typing Array Elements

TRAINING: typing $int[2][3]$

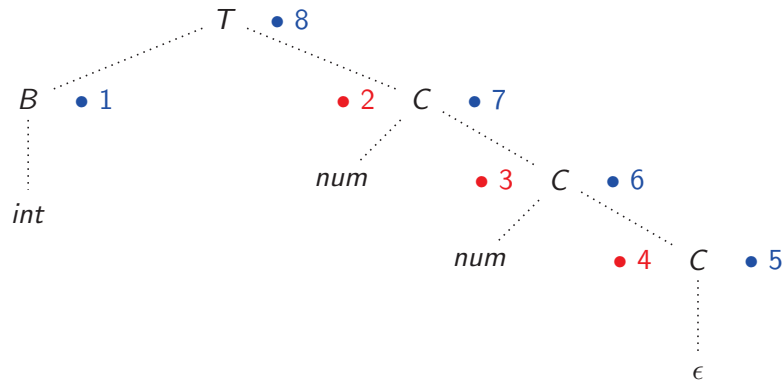


- 1: $B.t = integer$ from the semantic action for $B \rightarrow int$
- 2: $C.b = integer$ from the semantic action for $T \rightarrow BC$

63 / 85

Typing Array Elements

TRAINING: typing $\text{int}[2][3]$



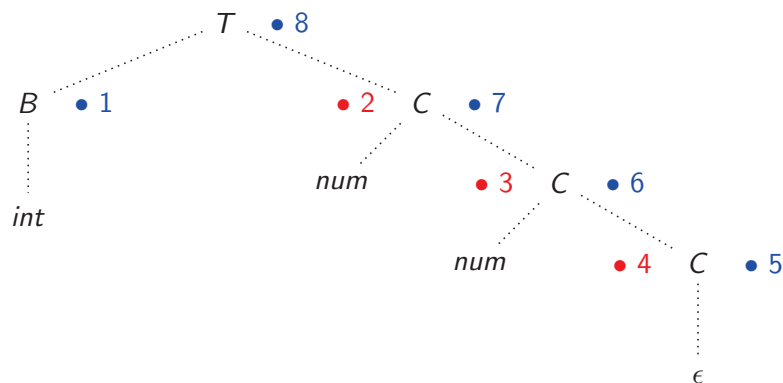
3: $C.b = \text{integer}$ from the semantic action for $C \rightarrow [\text{num}] C_1$

4: $C.b = \text{integer}$ from the semantic action for $C \rightarrow [\text{num}] C_1$

64 / 85

Typing Array Elements

TRAINING: typing $\text{int}[2][3]$



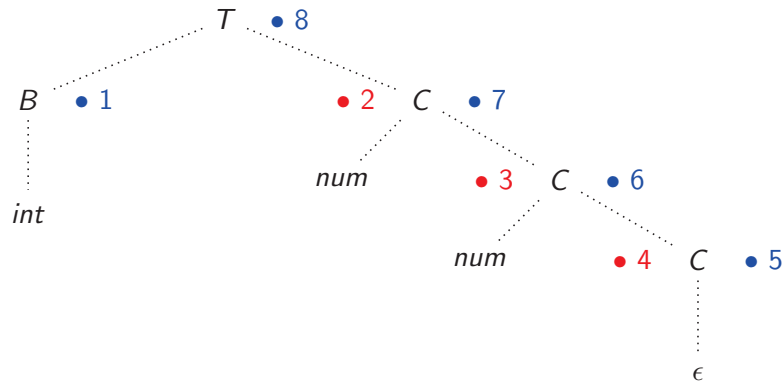
5: $C.t = \text{integer}$ from the semantic action for $C \rightarrow \epsilon$

6: $C.t = \text{array}(3, \text{integer})$ from the semantic action for $C \rightarrow [\text{num}] C_1$

65 / 85

Typing Array Elements

TRAINING: typing $\text{int}[2][3]$



7: $C.t = \text{array}(2, \text{array}(3, \text{integer}))$ from the semantic action for $C \rightarrow [\text{num}] C_1$

8: $T.t = \text{array}(2, \text{array}(3, \text{integer}))$ from the semantic action for $T \rightarrow BC$

66 / 85

Addressing Array Elements

ROW-MAJOR ORDER

- The main issue in generating code for arrays is the computation of the address of their elements
- Elements are stored in consecutive locations to ease accessing them
- If elements are $0, 1, \dots, n$, base is the relative address of the storage allocated for the one-dimensional array, and w is the width of each element, then the i th element starts at

$$\text{base} + i * w$$

- For two-dimensional arrays, if w_1 is the width of an entire row, and w_2 the width of an element in a row, then $A[i_1][i_2]$ starts at

$$\text{base} + i_1 * w_1 + i_2 * w_2$$

67 / 85

Syntax-directed translation of arrays

$$S \rightarrow id = E \mid L = E$$

$$E \rightarrow E + E \mid id \mid L$$

$$L \rightarrow id[E] \mid L[E]$$

To generate, e.g.,

- $a[b] = c$
- $a[b][c] = d[e]$
- $a = b[c]$

Syntax-directed translation of arrays

Attributes:

- $L.addr$ denotes a temporary used in computing the due offset
- $L.array$ is a reference to the symbol table entry for the array name; the entry contains the *base* address of the array ($L.array.base$) and the type of the array
- $L.type$ holds the type of L
- $L.width$ is the width of the subarray generated by L

Syntax-directed translation of arrays

$$S \rightarrow L = E$$

Generate an instruction to assign the value denoted by E to the location denoted by the reference L

$L.addr$ holds the offset from the base, which is in $L.array.base$

Hence the location for L is $L.array.base[L.addr]$

$$S \rightarrow L = E$$

$$gen(L.array.base '[L.addr]' '=' E.addr)$$

70 / 85

Syntax-directed translation of arrays

$$E \rightarrow L$$

$$E.addr = newtemp()$$

$$gen(E.addr '=' L.array.base '[L.addr]')$$

71 / 85

Syntax-directed translation of arrays

$L \rightarrow id[E]$	$L.array = table.get(id)$
	$L.type = arg2(table.getType(id))$
	$L.width = width(L.type)$
	$L.addr = newtemp()$
	$gen(L.addr '=' E.addr '*' L.width)$

72 / 85

Syntax-directed translation of arrays

$L \rightarrow L_1[E]$	$L.array = L_1.array$
	$L.type = arg2(L_1.type)$
	$L.width = width(L.type)$
	$t = newtemp()$
	$gen(t '=' E.addr '*' L.width)$
	$L.addr = newtemp()$
	$gen(L.addr '=' L_1.addr '+' t)$

73 / 85

Syntax-directed translation of arrays

TRAINING: translating $b = c + a[i][j]$

Assuming that the type of a is $\text{array}(2, \text{array}(3, \text{integer}))$, that c, i, j are integers, that base of a is 0 and that the width of integers is 4, translate

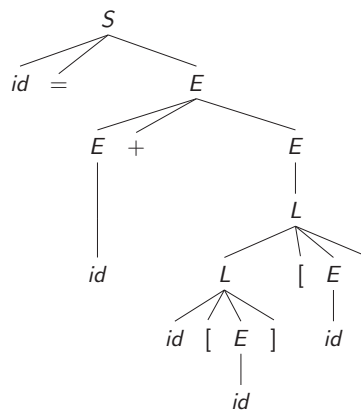
$$b = c + a[i][j]$$

74 / 85

Syntax-directed translation of arrays

TRAINING: translating $b = c + a[i][j]$

Derivation tree

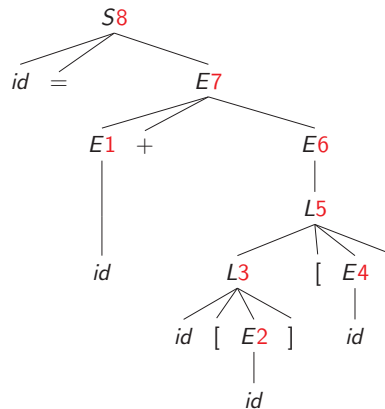


75 / 85

Syntax-directed translation of arrays

TRAINING: translating $b = c + a[i][j]$

All attributes are synthesised, evaluation can be carried out during bottom-up parsing (reduction steps ordered as shown)



76 / 85

Syntax-directed translation of arrays

TRAINING: translating $b = c + a[i][j]$

- | | |
|--|---|
| <p>1: $E.addr = c$</p> <p>2: $E.addr = i$</p> <p>3: $L.array = a$
 $L.type = array(3, int)$
 $L.width = 12$
 $L.addr = t_1$
 $t_1 = i * 12$</p> <p>4: $E.addr = j$</p> | <p>5: $L.array = a$
 $L.type = int$
 $L.width = 4$
 $t_2 = newtemp()$
 $t_2 = j * 4$
 $L.addr = t_3$
 $t_3 = t_1 + t_2$</p> <p>6: $E.addr = t_4$
 $t_4 = a[t_3]$</p> <p>7: $t_5 = c + t_4$</p> <p>8: $b = t_5$</p> |
|--|---|

77 / 85

Wrap up

78 / 85

Typical phases of a compiler

Lexical Analyser

Syntax analyser

Semantic analyser

Intermediate code generator

Code optimizer

Code generator

79 / 85

Lexical Analyser

$position = initial + rate * 60$



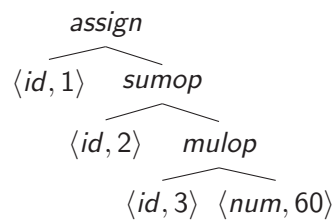
$\langle id, 1 \rangle \text{ assign } \langle id, 2 \rangle \text{ sumop } \langle id, 3 \rangle \text{ mulop } \langle num, 60 \rangle$

Symbol table

1	<i>position</i>	
2	<i>initial</i>	
3	<i>rate</i>	

Syntax Analyser

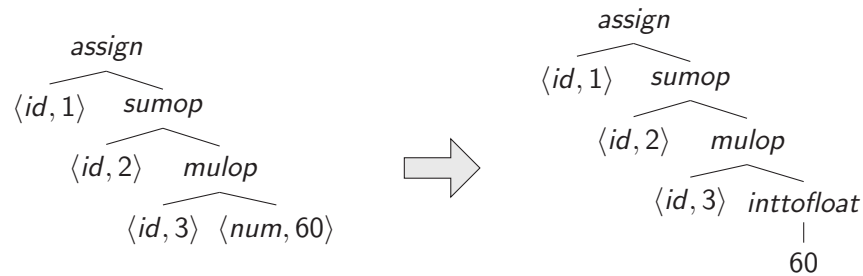
$\langle id, 1 \rangle \text{ assign } \langle id, 2 \rangle \text{ sumop } \langle id, 3 \rangle \text{ mulop } \langle num, 60 \rangle$



Symbol table

1	<i>position</i>	
2	<i>initial</i>	
3	<i>rate</i>	

Semantic Analyser

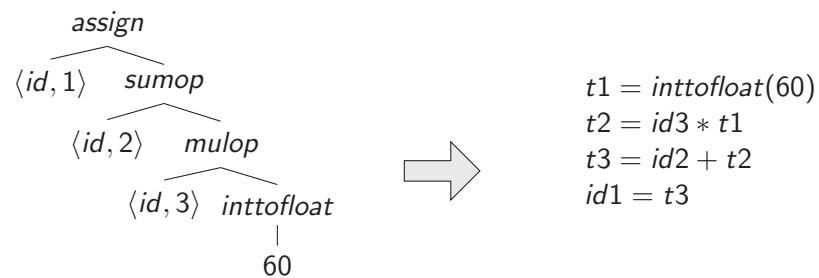


Symbol table

1	<i>position</i>	float	
2	<i>initial</i>	float	
3	<i>rate</i>	float	

82 / 85

Intermediate code generator



Where *id1*, *id2*, *id3* are the addresses of the current (scoping-wise) instances of the three identifiers

83 / 85

Code optimizer

$t1 = \text{inttofloat}(60)$		$t1 = id3 * 60.0$
$t2 = id3 * t1$		$id1 = id2 + t1$
$t3 = id2 + t2$		
$id1 = t3$		

84 / 85

Code generator

$t1 = id3 * 60.0$		<i>LDF</i> <i>R2</i> , <i>id3</i>
$id1 = id2 + t1$		<i>MULF</i> <i>R2</i> , <i>R2</i> , 60.0
		<i>LDF</i> <i>R1</i> , <i>id2</i>
		<i>ADDF</i> <i>R1</i> , <i>R1</i> , <i>R2</i>
		<i>STF</i> <i>id1</i> <i>R1</i>

85 / 85